

SPEC 2111

Deep Learning and Neural Networks

Course Roadmap

PART-1: Introduction, Foundations & Essentials

- Week 1 – Introduction & Course Overview
- Week 2 – Linear Regression
 - Regression concepts, gradient descent, loss functions
- Week 3 – Logistic Regression
 - Classification, sigmoid, cross-entropy loss

PART-2: Neural Networks, Deep Learning and Training

- Week 4 – Neural Network Essentials
 - Neurons, layers, forward propagation, activation functions
- Week 5 – Training, Refining Backpropagation and Deep Neural Networks (DNNs), Backpropagation, chain rule, gradient descent optimization
- Week 6 – Preventing Overfitting, Activation Functions, Optimizers

PART-3: Specialized Architectures

- Week 7 – Convolutional Neural Networks (CNNs)
 - Convolutions, pooling, image applications
 - Assessment-1
- **Week 8 – Representations & Transfer Learning** 
- Week 9 – Recurrent Neural Networks & LSTM
 - RNNs, LSTM/GRU, sequence modeling
- Week 10 – Attention & Transformers
 - Attention mechanism, transformer architecture, NLP applications

PART-4: Reinforcement Learning & Projects

- Week 11 – Reinforcement Learning
 - **Assessment-2**
- Week 12 - Project Presentations & Discussions (**Project Due**)

Course Roadmap (Remaining Weeks)

# Week	Date	Topic	Notes
Week 8	18th of March	Representations and Transfer Learning	
Week 9	25th of March	RNNs & LSTMs	
Spring Break	1st of April	NO CLASS	
Spring Break	8th of April	NO CLASS	
No Class	15th of April	NO CLASS	
Week 10	22nd of April	Attention & Transformers	
Week 11	29th of April	Reinforcement Learning	ASSESSMENT 2
Week 12	6th of May	Discussions on New Topics, Project Presentations*	PROJECTS DUE 4th of May@5pm

*Reach out if you cannot share the data for the project work to get a slot for presentations

Week-8 / Agenda

Theme: Representations & Transfer Learning

- Recap - Week 7
 - Residuals
 - Inception
- Representations and Transfer Learning
 - Text Representations
 - BOW and One Hot Encoding
 - Distributed Word Vectors and Word2Vec
 - Autoencoders
 - CNNs for Text
 - Transfer Learning
- Lab Work



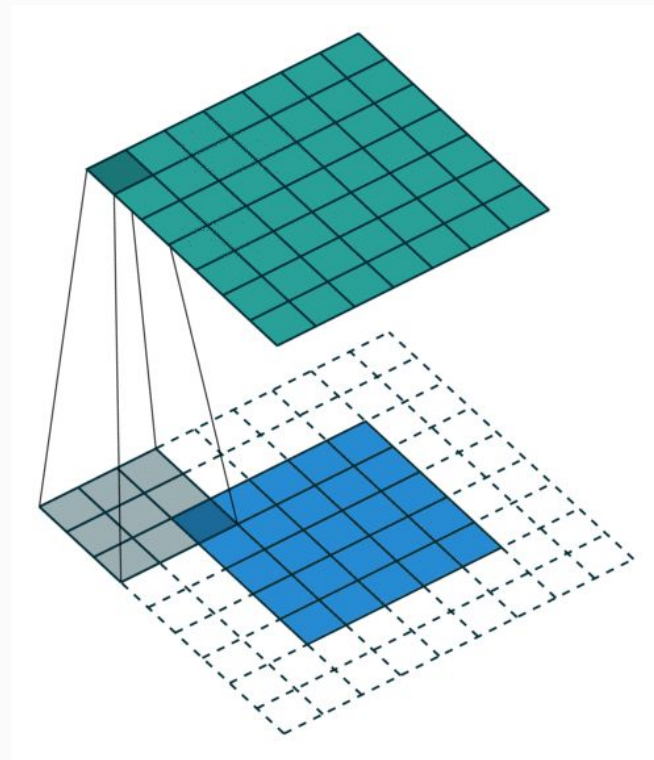
CNNs

CNNs are a type of deep learning model/architecture designed to automatically detect patterns in structured data by

- applying convolutional filters over local regions
- exploiting locality and translation invariance.

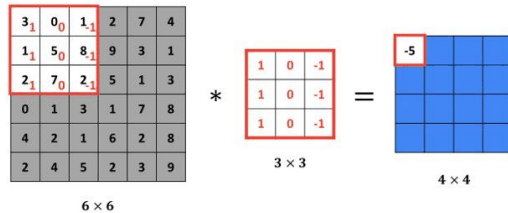
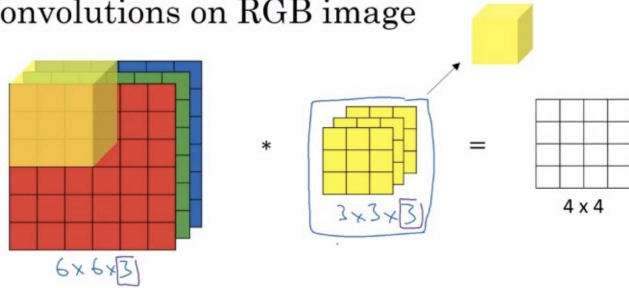
Key Features:

- **Locality:** Focuses on small neighboring regions (e.g., image patches, word sequences).
- **Translation Invariance:** Same filter detects features regardless of position.
- **Versatility:** Works on images, text, audio, and other structured data.



Convolution - Sliding a filter across the image

Convolutions on RGB image

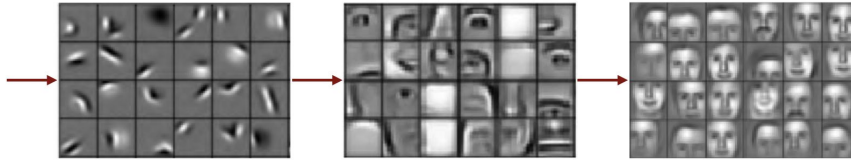


$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

TERMINOLOGY

- A **channel** is one layer of information in the data (used for both input, filters etc).
 - Example: a color image
 - Height $\times$ Width $\times$ Channels
 - 1 channel for red, 1 for blue, 1 for green
 - So the image is three stacked 2D matrices.
- A **filter** is a stack of kernels, one for each input channel, that together detect a feature across all channels and produce one output feature map.
 - Filter depth = input depth
- A **kernel** is a small matrix of weights that is applied to one input channel
- A **patch** is the small region of the input the filter is currently looking at.
- **Feature map** is the output of a filter after convolution across the entire input
- **Stride** controls the overlap between patches.
- **Padding** adds extra pixels around the edges of the input image (usually zeros).

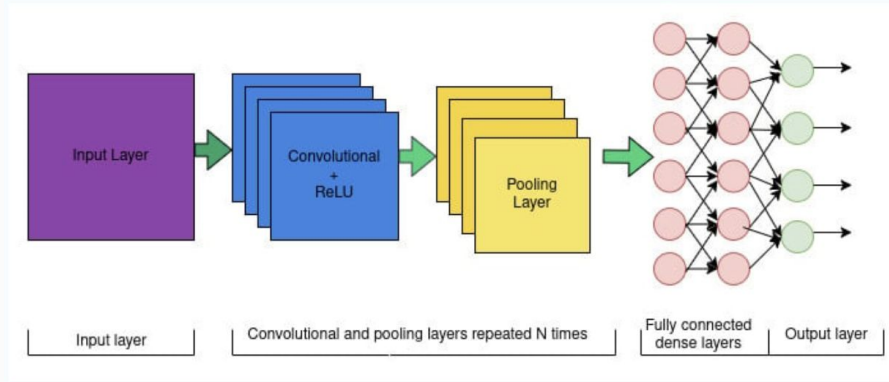
Later conv layers “see” more of the original input than the earlier layers



lines => edges, circles => faces!

- At first, the network detects small features like edges in tiny patches.
- As layers stack up, neurons “see” larger areas
- Eventually, the network can detect entire objects or large patterns, not just small edges.

CNN Architecture



CNN model components:

- Input layer
- Convolutional layers:
 - Convolutional layers are the specialty of CNN models.
 - Each convolutional layer applies a set of filters to detect features from the input images.
 - A CNN layer gives feature maps as its output.
- Pooling layers:
 - used to downsample the feature maps.
 - reduces the size of the feature matrix
 - summarizes the most prominent features in a particular region of the feature map while discarding less significant details.
- Fully connected layers:
 - After multiple convolutional and pooling layers, we flatten the feature maps and pass them to fully connected layers
- Output layer

Why Very Deep Networks Struggle

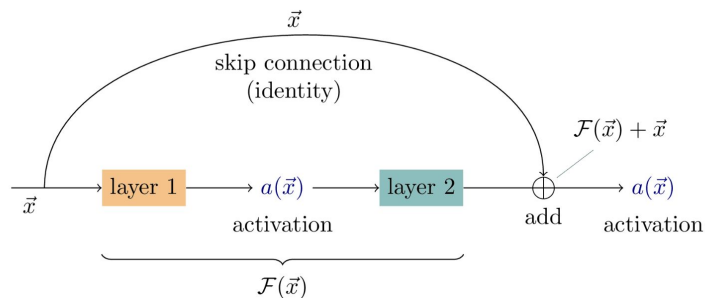
- Stacking more layers $\rightarrow$ can capture more complex features.
- Problem: Vanishing gradients
 - Gradients shrink as they backpropagate through many layers.
 - Early layers stop learning effectively.
- Deep CNNs were historically limited by this problem.

In a deep network, the gradient has to pass through many layers.

Each layer multiplies the gradient by some number (often < 1). After many multiplications:

gradient $\approx$ small $\times$ small $\times$ small $\times$ small $\rightarrow$ **almost zero**

So early layers barely learn.



Skip Connections as a solution

- Input bypasses one or more layers
 - added directly to later layer outputs.
- Enables deeper networks without vanishing gradient issues.

Skip Connections

What a skip connection changes

In a normal layer:

$$y = F(x)$$

In a residual (skip) block (used in ResNet):

$$y = x + F(x)$$

When we compute gradients:

$$\frac{\partial y}{\partial x} = 1 + \frac{\partial F(x)}{\partial x}$$

The important part is the "+1".

Without skip connection:

$x \rightarrow \text{layer} \rightarrow \text{layer} \rightarrow \text{layer} \rightarrow \text{layer} \rightarrow \text{loss}$

Gradient must survive **every layer**.

With skip connections:

$x \xrightarrow{\quad \quad \quad} (+) \rightarrow \text{loss}$
 $\quad \searrow \text{layer} \rightarrow \text{layer} \quad /$

Gradient has a **shortcut back to earlier layers**.

Think of a skip connection like a shortcut road:

- **main road** (convolution layers) learns complex transformations.
- shortcut road (**skip connection**) allows information to travel directly from input to output.
- Gradients can flow through both paths:
 - Through deep transformation $F(x)$
 - Through the direct identity path x
- Even if gradients shrink through $F(x)$, the identity path still carries gradient directly.
- +1 guarantees gradient flow.

Residual Layers

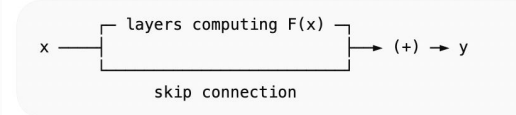
Residual block = standard layer(s) + skip connection



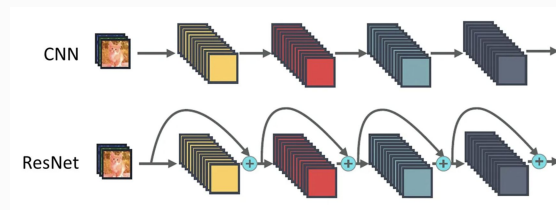
Then the **skip connection** adds the input:

$$y = x + F(x)$$

So the block looks conceptually like:



ResNet = stack of residual blocks



- Process:
 - Input $x \rightarrow$ Layer(s) $\rightarrow$ Activation $\rightarrow$ Output
 - Skip connection:
 - x added before activation at end of block
- Gradients can flow backward more easily $\rightarrow$ allows very deep CNNs.

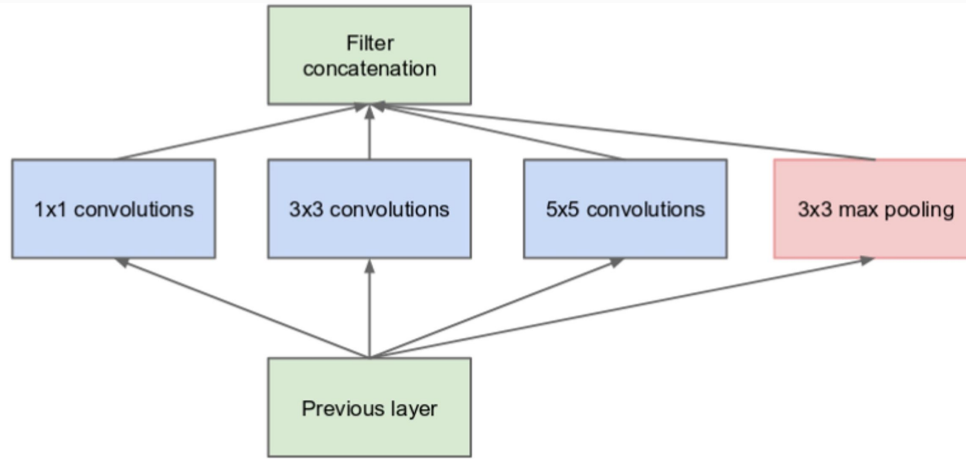
ResNet

If your neural network becomes **deep**, add **skip connections**.

- ResNet (Residual Network) is a particular type of CNN
 - Highly successful CNN architecture using residual learning.
- It is built entirely from residual blocks (layers + skip connections).
- Mainly for images! As for images we generally need deeper networks
- Unlike standard CNNs:
 - ResNet can be very deep (50, 101, 152+ layers)

Without skip connections, CNNs were typically 10–20 layers deep at most.
With residuals, networks can now go much deeper, 50, 101, 152+ layers.

Inception and other variants



(a) Inception module, naïve version

- Standard CNNs used one kernel size per layer, limiting scale flexibility.
- But objects in images appear at different scales.
 - Example: small eyes, large object
- A single kernel size might not capture all scales effectively.
- **Inception Network solve this problem.**
 - Instead of choosing one kernel size, they use multiple kernel sizes in the same layer.

Inception

- Inception networks make CNNs wider instead of just deeper.
- Multiple filters per layer → flexible and richer feature extraction.
- Because features are detected at multiple sizes, the network can recognize objects even if their size changes.
- Great for complex visual scenes

The trade-off of using Inception Network is mainly higher computational cost and architectural complexity compared with a standard CNN layer

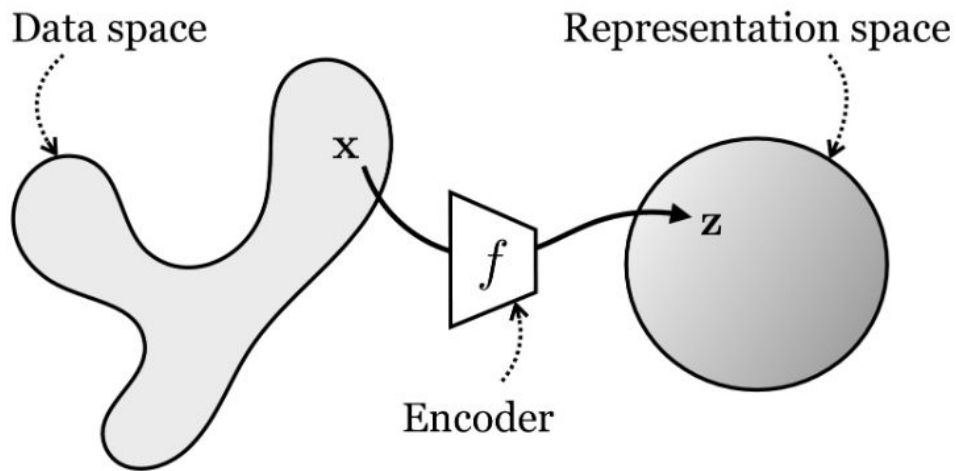
ResNet & Inception

Model	Key Idea	Use Case
ResNet	Skip connections (deep)	Very deep networks, stable training
Inception	Multi-scale filters (wide)	Objects at different sizes

CNN Architectures in Keras: ResNet vs Inception

- Do they exist out-of-the-box?
 - Both available as pre-built models in TensorFlow / Keras:
 - ResNet50, ResNet101, InceptionV3, InceptionResNetV2
 - These are full networks, not single layers
- Is there a single “ResNet/Inception layer”?
 - No, they are architectural blocks, not standalone layers
 - You must implement manually if building from scratch:
 - ResNet → skip connections (Add), Inception → parallel convs + Concatenate

Representations



What Do We Mean by a Representation?

- A representation is simply how information is encoded.
- For humans, information may appear as images, sounds, or language.
 - But neural networks operate on numbers.
 - So we must convert real-world data into numerical form that a model can process.
- A good representation should:
 - Preserve important information
 - Be efficient for computation
 - Allow models to learn patterns
 - Capture relationships between data
- Poor representations can make learning very difficult

Converting real-world information into numbers.

Real World	Representation
Image	Pixel matrix
Audio	Waveform / spectrogram
Text	Word vectors

Example: Image Representation

- Images are actually very convenient for machine learning.
- A digital image is already stored as numbers — pixels.
- This means we can feed pixel values directly into neural networks such as CNNs.
- The representation naturally preserves spatial structure.



```

0 0 7 15 0 0 11 10 0
0 0 0 4 0 102 126 256 255 0 37 0 0 0 0 2
0 0 10 16 11 238 255 244 244 242 249 255 22 21 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
255 228 255 255 255 255 255 255 255 255 255 255 255 255 255 255
13 117 43 243 255 13 12 26 52 2 0 0 0 232 255 255 4
16 255 255 255 45 12 0 0 7 7 0 0 232 255 255 255
0 255 255 255 255 255 11 0 0 0 0 0 238 243 255 255
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 13 15 255 255 255 255 255 255 255 255 255 255 255 255 0
1 0 0 0 1 255 255 255 255 255 255 255 255 11 37 0 0
0 0 0 4 0 126 255 255 255 255 255 255 255 11 0 0
0 2 208 255 255 255 255 255 255 255 255 255 255 255 304 9
0 0 255 255 255 255 255 24 0 0 0 0 255 255 255 255 255
0 18 255 255 255 7 11 0 0 0 2 255 255 255 255
0 255 255 255 0 10 20 0 33 3 255 255 255 255 255
0 0 255 255 24 255 255 08 05 10 255 255 255 255 255
0 0 255 255 255 255 255 255 255 255 255 255 255 255 255
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

```

```

0 0 2 15 0 0 11 10 0 0 0 0 0 0 0 0
0 0 0 0 60157 236 255 177 95 31 0 0 29
0 0 16119 238 255 244 243 250 249 255 222 103 0
0 14070 255 244 254 255 253 245 245 253 251 124 1
2 9856 228 255 251 254 251 141 116 112 215 251 238 95
13217 243 255 155 3226 52 2 0 10 13322 255 637
16229 252 254 49 12 0 0 0 70 237 252 253 637
6141 245 255 212 25 11 9 3 0 115 236 243 255 137
0 87752 254 248 215 60 0 1 121 252 255 248 66 0
0 13130 255 245 255 182 181 248 252 242 208 36 0 19
0 1 511 257 255 241 255 247 255 241 162 17 7 0
0 0 4 58 251 255 244 253 235 255 120 11 0 1
0 4 97 255 255 245 248 252 254 255 182 10 4
0 2206 252 246 251 241 100 0 14 235 245 255 194 9
01181 255 245 158 24 0 0 6 39 255 232 250 56
02756 251 250 137 7 11 0 0 2 62255 250 125 3
0173 255 101 9 20 0 13 3 182 251 245 61
0107 251 243 255 320 95 55 19 118 217 255 255 42
0 18146 250 257 245 255 255 249 255 240 255 129 0
0 0 23131 215 254 246 255 255 248 248 118 14 0
0 0 6 1 0 52153 233 255 252 147 37 0 0 4 1
0 0 5 5 0 0 0 0 14 1 0 6 6 0

```

Text Representations

word	The	cat	sat	on	the	mat.
the	1	0	0	0	1	0
cat	0	1	0	0	0	0
on	0	0	0	1	0	0
⋮						

$N_{\text{unique_words}}$

Text Representation

- Text presents a bigger challenge.
 - unlike images, text is not naturally structured for machine learning.
- A neural network cannot directly understand the string "dog".
 - the first problem in NLP is designing a representation
 - that converts words into numbers
 - while preserving useful information.
- Words vary in length and meaning.
- Even more importantly, word order changes meaning.
 - Example:
 - "John gave Mary €500"
 - "Mary gave John €500"
 - Same words → different meaning.
- So any representation must try to capture both content and structure.

Before converting text to numbers, common preprocessing steps

Steps before any numerical representation to have consistent, meaningful, and processable text:

1. Tokenization – split text into words, subwords, or characters
2. Lowercasing / Normalization – make text consistent (Dog → dog)
3. Removing punctuation / special characters
4. Stopword removal – remove common words if desired (the, is)
5. Stemming / Lemmatization – reduce words to base forms (running → run)
6. Handling unknown words – replace rare/unseen words with UNK

Original text:

"Running dogs are amazing! The dog ran quickly."

1. **Tokenization** – split text into words/subwords/characters

Word-level: ["Running", "dogs", "are", "amazing", "!", "The", "dog", "ran", "quickly"]
Subword-level: ["Run", "##ning", "dog", "##s", "are", "amaz", "##ing", "!"]
Character-level: ["R", "u", "n", "n", "i", "n", "g", " ", "d", "o", "g", "s", "..."]

2. **Lowercasing / Normalization** – make text consistent

"running dogs are amazing! the dog ran quickly."

3. **Remove punctuation / special characters** (optional)

"running dogs are amazing the dog ran quickly"

4. **Stopword removal** (optional)

["running", "dogs", "amazing", "dog", "ran", "quickly"]

(removed: "are", "the")

5. **Stemming / Lemmatization** (optional)

["run", "dog", "amazing", "dog", "run", "quickly"]

- Stemming: running → run
- Lemmatization: ran → run

6. **Handling unknown words**

Replace rare/unseen words with UNK token
["run", "dog", "amazing", "dog", "run", "UNK"]

Early Text Representation: Bag of Words

- One of the simplest representations is Bag of Words (BOW).
- Idea:
 - BoW represents an entire document using a vocabulary.
 - Each feature corresponds to a word in the vocabulary.
 - Values can be:
 - Binary (0 = absent, 1 = present)
 - Counts (number of occurrences)
- Example:
 - Vocabulary: [cat, dog, fish]
 - Document: "cat fish fish"
 - BoW (count): [1, 0, 2]
 - BoW (binary): [1, 0, 1]

Vocabulary: [the, red, dog, cat, eats, food]

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

Despite its simplicity, it worked surprisingly well for tasks like spam detection and document classification.

- Document/Sentence = a single row in the table
- Dataset = the whole table

Problems with Bag of Words

- Word order lost
 - "John loves Mary"
 - "Mary loves John"
 - Same representation
- Huge sparse vectors
 - Vocabulary may be 50k+ words
- No semantic similarity
 - "Dog"
 - "Labrador"
 - Completely unrelated in BOW space.

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

From Bag of Words to Word Level Representations

- Represent each word individually.
- Now the document becomes a matrix rather than a single vector.
- This **preserves word order**, which is critical for language.
- One Hot Encoding
 - Each word is represented as a one-hot vector (length = vocab size)
 - A sentence or document becomes a matrix (number of words × vocab size)
 - This preserves word order in the sequence

Example:

Vocabulary = [cat, dog, fish]

Sentence = "cat fish fish"

Word	One-Hot Vector	Resulting document matrix:
cat	[1, 0, 0]	[[1, 0, 0], [0, 0, 1], [0, 0, 1]]
fish	[0, 0, 1]	
fish	[0, 0, 1]	

Matrix shape = (num_words × vocab_size)

Word order is preserved because

- row 1 = first word,
- row 2 = second word, etc.

Problems with One Hot Encoding

Sparsity

One-hot vectors are very sparse.

- If the vocabulary size is 10,000 words, each vector has 10,000 dimensions.
- Only one value is 1, and the rest (9,999) are 0.

This causes:

- High memory usage
- Inefficient computation

No Semantic Meaning

One-hot encoding does not capture relationships between words.

Example:

- cat $\rightarrow [1,0,0,0]$
- dog $\rightarrow [0,1,0,0]$
- car $\rightarrow [0,0,1,0]$

Distance between cat and dog = same as cat and car

But cat and dog are semantically similar, while cat and car are not

There's no connection between the meaning of a word and its one-hot vector!!

From One-Hot Encoding to Distributional Semantics

- One-hot encoding is simple but sparse and independent.
- Instead if you think about words and language:
 - meaning of a word can be derived from the distribution of contexts in which it appears.
 - Distributional hypothesis
 - We can learn about the meaning of a word by investigating the distribution of words that show up around the word
 - “You shall know a word by the company it keeps!” J.R. Firth (1957)
 - “The meaning of words lies in their use” Ludwig Wittgenstein (1953)
- Distributional Hypothesis implies that words that appear in similar “contexts” will share similar meanings
- This simple (and old) idea is one of the most influential and successful ideas in modern NLP
- Word embedding approaches represent the distributional “meaning” of a word as a vector in multidimensional space

Move from sparse, independent one-hot vectors → dense, semantic vectors

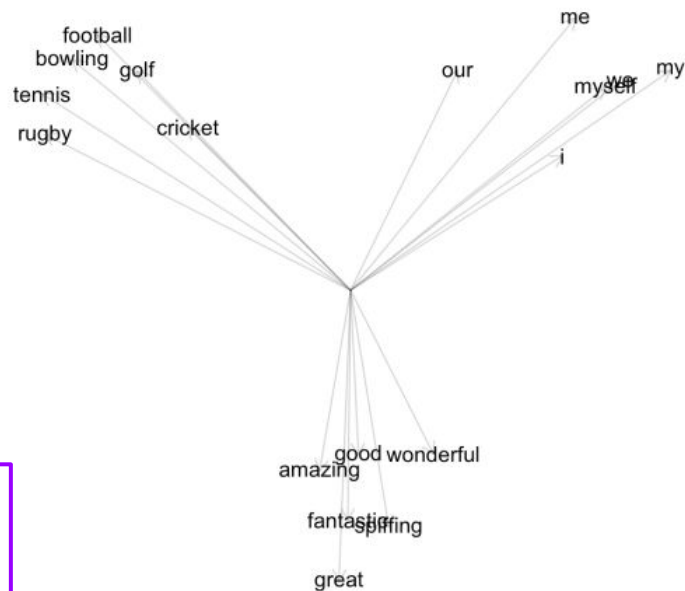
Contextual embeddings capture both:

- Meaning (semantic relationship)
- Word order / context

From One-Hot Encoding to Distributional Semantics

- The meaning of each word is based on the distribution of terms with which it co-occurs
 - We represent this meaning using a vector for each word
 - Vectors are constructed such that similar words are close to each other in “semantic” space
 - We build this space automatically by seeing which words are close to one another in several texts
 - known as word embeddings because we “embed” words into a low-dimensional space

- In embedding space:
 - Related words cluster together.
- Distance between vectors represents semantic similarity.
 - Example:
 - $\text{distance}(\text{football}, \text{tennis}) < \text{distance}(\text{Football}, \text{wonderful})$



How Do We Learn Word Embeddings?

We can learn embeddings by training a neural network to predict words.

IDEA: Predict the word from context!

Step-1: Create training examples from text

- Start with sentences from a large corpus.
- Choose a context window (e.g., window size = 1). This means we look at 1 word to the left and right of a target word.
 - Ex: the dog chased the ball
- Training pairs become:

Input (target word)	Output (context word)
dog	the
dog	chased
chased	dog
chased	the
.....	

So from a sentence we generate many input–target pairs.

2. Convert words to vectors

Example vocabulary:

Word	One-hot
dog	[1,0,0,0]
chased	[0,1,0,0]
ball	[0,0,1,0]
cat	[0,0,0,1]

The **input to the network** is the one-hot vector.

3. Train a model to predict output from input

Input (one-hot):

dog → [1,0,0,0]

Hidden layer:

embedding(dog) → [0.21, -0.34, 0.78]

Output (softmax probabilities):

[0.05, 0.60, 0.10, 0.25]

Target:

chased → [0,1,0,0]

How Do We Learn Word Embeddings?

Input word



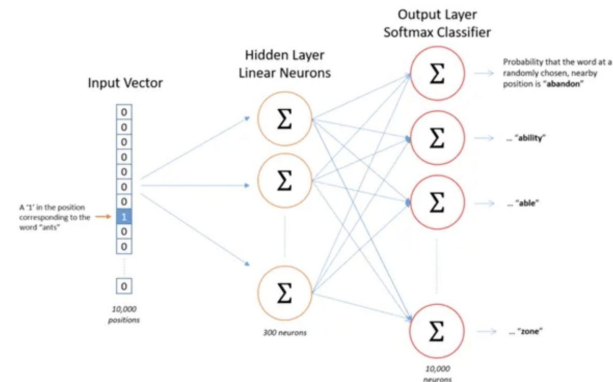
Embedding layer (creates word vector)



Prediction layer



Predicted context word



The first hidden layer becomes the embedding because:

- Words that appear in similar contexts must behave similarly for the model to succeed.
- Therefore words with similar contexts would result in same predictions in the output layer.
- For that, they should have same or similar weights in the hidden layer.

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 8 & 2 & 1 & 9 \\ 6 & 5 & 4 & 0 \\ 7 & 1 & 6 & 2 \\ 1 & 3 & 5 & 8 \\ 0 & 4 & 9 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 5 & 8 \end{bmatrix}$$

One-hot vector

Embedding Weight Matrix

Hidden layer output

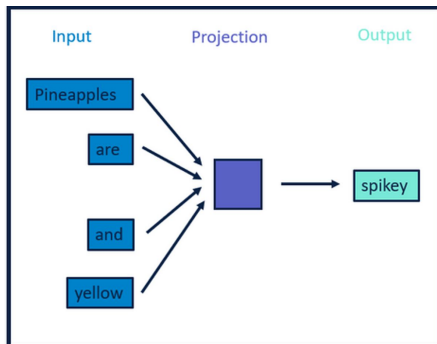
When you multiply a word's one-hot encoded vector with the embedding matrix (look up table), you get the embedding vector for the word.

Word2Vec - Efficient Word Embeddings

- Problem with basic embedding networks
 - Simple embedding networks can learn word vectors, but:
 - Inefficient for large vocabularies
 - Each training step computes probabilities over all words in the vocab → slow
 - Hard to scale to large corpora
- Word2Vec Innovations
 - Scalable training techniques:
 - Hierarchical softmax → speeds up probability computation
 - Represent the vocabulary as a binary tree
 - Each leaf = a word in the vocabulary
 - Predict a word by walking from root to leaf
 - At each node, compute a binary probability (go left or right)
 - Total probability = product of probabilities along the path
 - Negative sampling → train with a few “negative” words instead of full vocab
 - Not predict for all words in vocabulary
 - predict context words within a chosen window as positive targets and a few randomly sampled words outside the context as negative targets to efficiently learn embeddings.
 - Efficient Representation
 - Embedding dimension is small (50–300) → dense vector, unlike huge one-hot vectors.

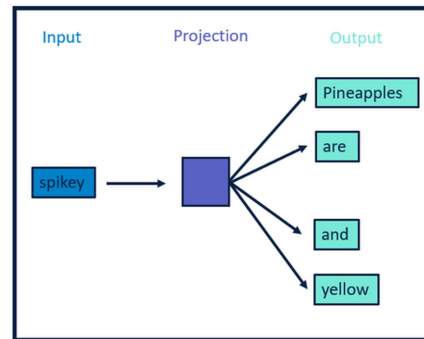
Word2Vec Variants

Contextual Bag of Words (CBOW)



- predicts the current word given context words within a specific window.
- input layer contains the context words and
- output layer contains the current word.
- CBOW treats context as a bag, so word order is ignored
- Captures **coarse semantic relationships**.
- **Faster** to train (averages multiple context words).

Skip Gram



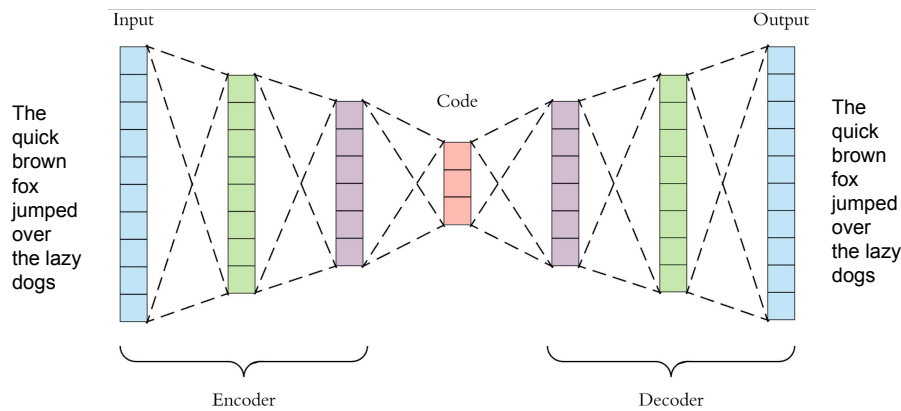
- predicts the surrounding context words within specific window given current word.
- input layer contains the current word
- output layer contains the context words.
- Captures **fine-grained semantic relationships**.
- **Slower** to train (more pairs per sentence).

From Word2Vec to Autoencoders for Word Embeddings

- Autoencoders offer an alternative to Word2Vec
- Instead of predicting context words (Word2Vec), use a neural network to reconstruct the input itself
- Essentially, treat word embedding learning as a reconstruction problem

Architecture

- Input: One-hot vector for a word
- Encoder: Compress input into a dense hidden layer (the embedding)
- Decoder: Reconstruct original input or context
 - Target same as the input
 - bottleneck hidden layer must capture all information needed to recreate the input → this becomes the word embedding.



Autoencoders

```
import numpy as np
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

# Example vocabulary of 5 words
vocab_size = 5
embedding_dim = 3 # size of embedding (bottleneck)

# Example one-hot encoded words
# "cat", "dog", "fish", "bird", "cow"
X = np.eye(vocab_size) # identity matrix as one-hot vectors

# --- Autoencoder architecture ---
input_layer = Input(shape=(vocab_size,)) # one-hot input
encoded = Dense(embedding_dim, activation='relu')(input_layer) # bottleneck (embedding)
decoded = Dense(vocab_size, activation='softmax')(encoded) # reconstruct input

autoencoder = Model(inputs=input_layer, outputs=decoded)
autoencoder.compile(optimizer='adam', loss='categorical_crossentropy')

# --- Train autoencoder ---
autoencoder.fit(X, X, epochs=500, verbose=0)

# --- Extract embeddings from hidden layer ---
encoder = Model(inputs=input_layer, outputs=encoded)
embeddings = encoder.predict(X)

# Display embeddings
for i, emb in enumerate(embeddings):
    print(f"Word {i} embedding: {emb}")
```

- decoded = output layer of the network
- in an autoencoder, X is both the input and the target
- network learns embeddings by compressing X in a hidden layer and reconstructing it at the output.

- embedding vector = activation of the bottleneck layer for that input
 - Keras doesn't give you activations automatically unless you either:
 - Build a separate model from input → bottleneck, or
 - Use a functional hook / intermediate layer output

- Word2Vec is ideal for efficient word embeddings from large corpora
- autoencoders are more general, flexible, and can learn embeddings from any data (text, video, image, etc) by reconstructing the input.

Building & Using Word Embeddings in Practice

Word embeddings can be build & used in two main ways in real NLP systems.

Option 1 — Pre-trained/Train Embeddings

- First we train embeddings on a large text corpus using a model such as Word2Vec.
- Example:
 - dog → [0.21, -0.34, 0.78]
 - cat → [0.18, -0.30, 0.75]
 - car → [-0.66, 0.44, -0.12]
- Then we use these vectors as input features for another task.
 - Example tasks:
 - sentiment analysis
 - document classification
 - question answering
 - recommendation systems

Option 2 — Learn Embeddings During the Task

- Instead of using pre-trained embeddings (like Word2Vec or GloVe), let the model learn embeddings directly from your dataset while training for the specific task.
- Example task: Sentiment analysis on movie reviews
 - Build a network for this task
 - Embeddings are usually the weights of the first hidden layer (Embedding Layer)
 - Can be extracted after training
- Frameworks like TensorFlow and Keras make this very easy using an Embedding Layer.

Option 1 — Pre-trained Embeddings

- You can use Word2Vec as the training method for learning word embeddings.
 - It uses a neural network to learn vectors based on word context.
- Word2Vec is commonly implemented in libraries like:
 - Gensim (very popular)
 - TensorFlow
 - PyTorch

```
from gensim.models import Word2Vec

sentences = [
    ["the", "dog", "chased", "the", "ball"],
    ["the", "cat", "chased", "the", "mouse"],
    ["the", "dog", "barked"],
    ["the", "cat", "meowed"]
]

model = Word2Vec(
    sentences,
    vector_size=50, # embedding dimension
    window=2,      # context window size
    min_count=1,   # keep words appearing at least once
    sg=1           # 1 = skip-gram, 0 = CBOW
)

vector = model.wv["dog"]
print(vector)
```

In Word2Vec, you can feed tokenized words directly, and each word automatically gets a dense embedding from the internal lookup table.

Example output (shortened):

[0.12 -0.33 0.41 0.08 ...]

This is the **embedding vector** for "dog".

Word2Vec can find **semantic similarity**.

```
model.wv.most_similar("dog")
```

Example output:

```
[('cat', 0.78), ('barked', 0.63)]
```

Meaning:

Words that appear in **similar contexts** are close in vector space.

Option 2 — Learn Embeddings During the Task

- Example Task: Predict movie review sentiment from the IMDB Movie Reviews Dataset
 - Input: review text
 - Output: sentiment (positive / negative)
- **Step 1 — Tokenization**
 - Convert words → integer indices.
 - Example sentence: the dog chased the ball
 - Vocabulary mapping:
 - the → 1, dog → 2, chased → 3, ball → 4
 - Encoded sentence: [1, 2, 3, 1, 4]
 - libraries like TensorFlow and Keras usually skip one-hot encoding and use integer indices directly!
- **Step 2 — Padding**
 - Neural networks require fixed input length.
 - Example: Review 1: [1,2,3,4], Review 2: [5,2]
 - Pad shorter sequences with zeros:
 - [1,2,3,4,0]
 - [5,2,0,0,0]
 - Now all inputs have the same length.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Flatten, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

vocab_size = 10      # Number of unique words in vocabulary (for simplicity)
embedding_dim = 8    # Size of the embedding vector for each word
maxlen = 5           # Maximum sentence length
batch_size = 2       # Number of sentences in a batch
```

```
# Step 1: Tokenized example sentences
# -----
# Normally, you'd convert text to integers using a tokenizer
# Example sentences (tokenized):
# 'the dog chased the ball' → [1, 2, 3, 1, 4]
# 'cat sat' → [5, 6] (padded to maxlen)

sentences = [
    [1, 2, 3, 1, 4],
    [5, 6]
]

# -----
# Step 2: Pad sentences to fixed length
# -----
X = pad_sequences(sentences, maxlen=maxlen, padding='post', value=0)
print("Padded sentences: \n", X)
```

```
[[1,2,3,1,4],
 [5,6,0,0,0]]
```

Option 2 — Learn Embeddings During the Task

```
# -----  
# Step 2: Build embedding model  
# -----  
model = Sequential()  
model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=maxlen))  
model.add(Flatten())  
model.add(Dense(200, activation='relu'))  
model.add(Dense(1, activation='sigmoid'))  
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])  
  
# -----  
# Step 3: Dummy labels for training  
# -----  
y = np.array([1, 0])  
  
# Train briefly (for demonstration)  
model.fit(X, y, epochs=5, batch_size=2, verbose=0)  
  
# -----  
# Step 4: Inspect embeddings  
# -----  
embedding_layer = model.layers[0] # First layer is embedding  
weights = embedding_layer.get_weights()[0] # Shape: (vocab_size, embedding_dim)  
  
print("Embedding vector for 'dog' (index 2):", weights[2])  
print("Embedding vector for 'cat' (index 5):", weights[5])  
  
# -----  
# Step 5: Cosine similarity  
# -----  
def cosine_similarity(vec1, vec2):  
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))  
  
sim = cosine_similarity(weights[2], weights[5])  
print(f"Cosine similarity between 'dog' and 'cat': {sim:.3f}")
```

Embedding is a layer type, just like:

- Dense layer
- Convolution layer
- Pooling layer

So it is a special type of hidden layer designed specifically for text / categorical inputs.

Limitations of learning embeddings during task

1. Less Semantic Richness

- The embeddings may capture **task-specific patterns** but not general meaning.
- For example, "dog" and "labrador" might not end up close if your dataset never uses them interchangeably.

2. Limited Transferability

- These embeddings are not usually useful for **other tasks**.
- If you switch to a new task (e.g., question answering), you'd need to retrain embeddings.

Combine Pretrained Embeddings with Task Specific Networks

Train Word2Vec embeddings



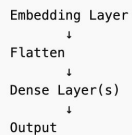
Load embeddings into Embedding layer



Train neural network for the task

Averaging vs Flattening

Flattening



```
model = Sequential()
model.add(Embedding(max_features,
                    embedding_size,
                    input_length=maxlen))
model.add(Conv1D(25, 3, padding='valid', activation='relu', strides=1))
model.add(Flatten())
model.add(Dense(200, activation='relu'))
model.add(Dense(1))

model.summary()
```

- Flattened vector: `[0.2, 0.5, -0.1, 0.1, 0.3, 0.4, 0.4, 0.0, 0.2]`
- Order of words is **preserved**

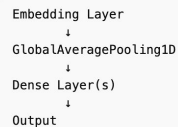
Pros:

- Keeps word order information
- Dense vector contains all info from all words

Cons:

- **Vector length grows** with sentence length → more parameters
- Slower training

Averaging



```
model = Sequential()
model.add(Embedding(max_features,
                    embedding_size,
                    input_length=maxlen))
model.add(Conv1D(25, 3, padding='valid', activation='relu', strides=1))
model.add(GlobalAveragePooling1D())
model.add(Dense(200, activation='relu'))
model.add(Dense(1))

model.summary()
```

`[0.2, 0.5, -0.1] + [0.1, 0.3, 0.4] + [0.4, 0.0, 0.2] = [0.7, 0.8, 0.5]`
Average → `[0.233, 0.267, 0.167]`

- Sentence: 3 words, embedding size = 3

```
dog → [0.2, 0.5, -0.1]
bites → [0.1, 0.3, 0.4]
man → [0.4, 0.0, 0.2]
```

Pros:

- Fixed-size vector regardless of sentence length
- Fewer parameters → faster training

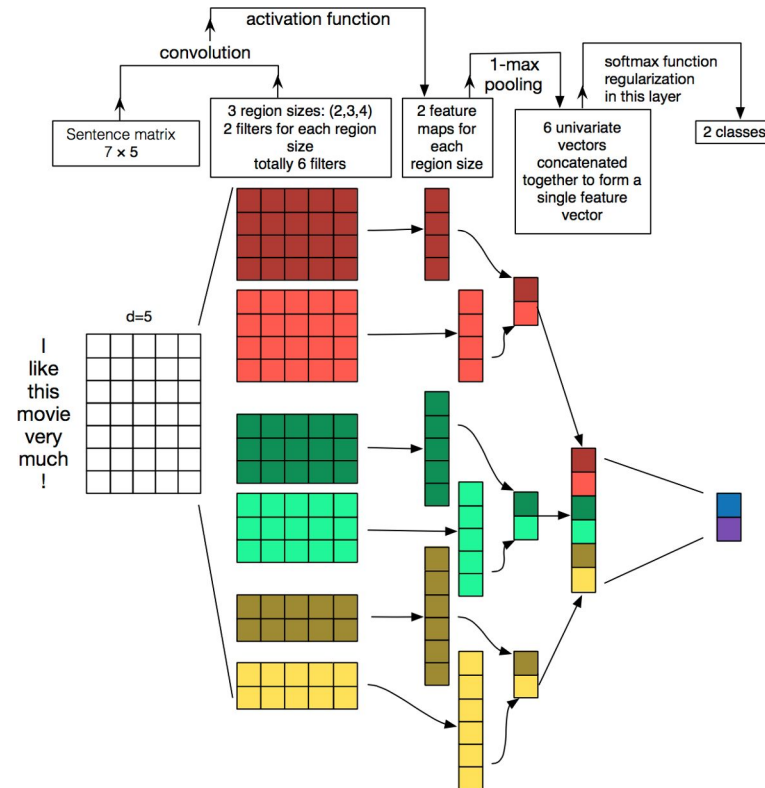
Cons:

- **Word order ignored**
- "dog bites man" ≈ "man bites dog"

CNNs for Text

- captures local context and sequence
- apply filters (feature detectors) over small groups of words.
- Example: A filter may look at 4 consecutive words at a time.
 - Think of filters as “pattern detectors” scanning through text.
 - Each filter learns a useful phrase pattern for the task
- For CNNs on text, your dataset is conceptually 3D:
 - (number_of_sentences, sentence_length, embedding_dim)
 - number_of_sentences → total sentences in your dataset
 - sentence_length → number of words in the sentence
 - embedding_dim → size of each word embedding
- CNNs require tensors to have the same shape in a batch.
 - Common solution: padding
 - Shorter sentences → pad with zeros (or a special <PAD> token)
 - Longer sentences → truncate to the max length

Example: sentiment classifier/2 classes

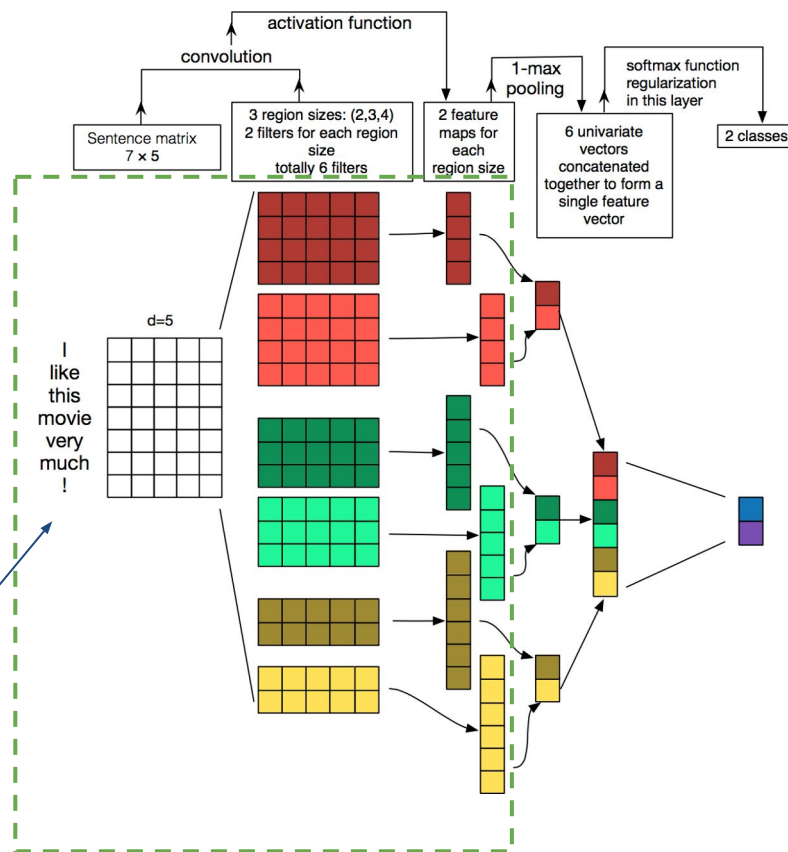


Convoluting through Embeddings – 1D Features

Feature Maps

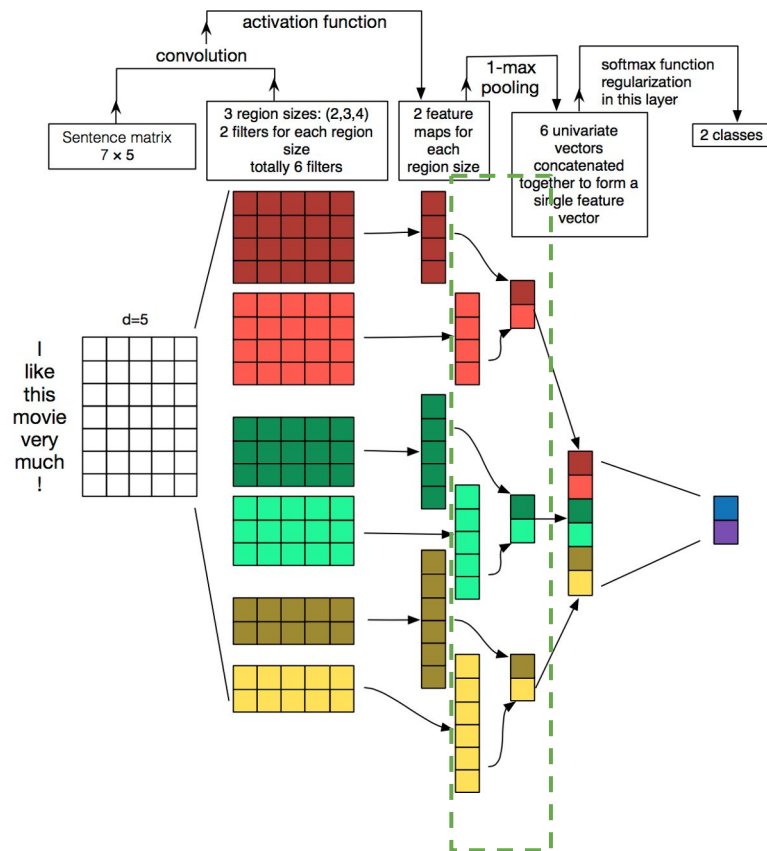
- Each filter produces a feature map
- Key Difference from Images:
 - Image CNNs convolve across height and width.
 - Text CNNs convolve only vertically (through words).
 - So you move across sequence of words, not embedding dimensions
- In text CNNs: Feature maps are 1D (not 2D like images)

- Document length = 7
- Embedding dimension = 5
- Filters:
 - 2 filters (size 2)
 - 2 filters (size 3)
 - 2 filters (size 4)
- Output: 6 feature maps



Pooling and Fully Connected Layer

- Apply **max pooling** to feature maps
 - Reduces each feature map to:
 - Either smaller size
 - Or a single value (global max pooling)
- Purpose:
 - Keep the most important feature
 - Reduce computation and overfitting
- Then feed into **fully connected layer and output layer**



When CNNs are good for text

CNNs work well for:

- Sentiment analysis
- Text classification
- Spam detection

They're especially strong at:

- Capturing local patterns (phrases like "not good", "very happy")
- Being fast and efficient

CNN vs Transformers (quick intuition)

- CNN → good at pattern detection in short context
- Transformers → good at understanding full context and relationships

Limitations

CNNs are not ideal when:

- You need long-range dependencies (understanding context across long paragraphs)
- Tasks require deep language understanding

That's why models like:

- RNNs (LSTM/GRU)
- Transformers (like BERT, GPT)

are often preferred for more complex NLP tasks.

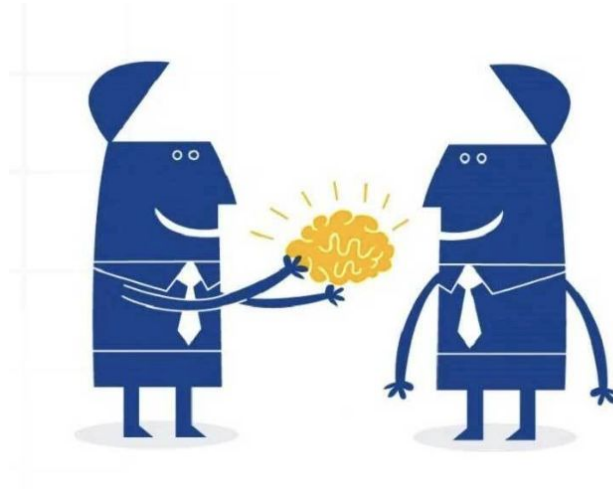
Re-Using Word / Text Embeddings

- There are pre-trained embeddings available for all major state-of-the-art embeddings generating architectures.
 - widely used because they are trained on large-scale datasets, so they capture robust representations.
- Many libraries like Hugging Face Transformers and Torchvision provide easy access to these embeddings.
- Examples
 - Word2Vec
 - Training Data: Google News dataset (~100B words)
 - GPT Embeddings (OpenAI GPT models)
 - Training Data: A mixture of web pages, books, Wikipedia, code, and other publicly available text

Knowing the source data helps understand the biases and strengths of each embedding.

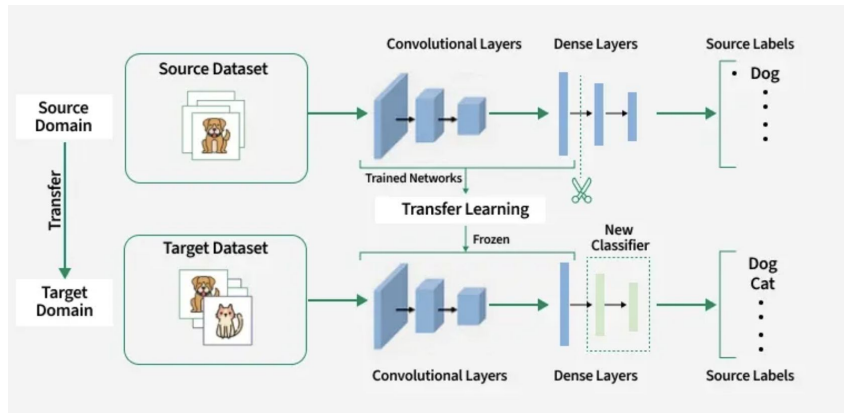


Re-Use and Transfer Learning



Transfer Learning

- Transfer Learning = using a model trained in one domain for a different domain
 - Typical workflow:
 - Train on large dataset
 - Reuse general-purpose layers
 - Replace and train new task-specific head
 - Head = final layers of a neural network that are specific to the original task.
 - Usually includes: Fully connected layers and output layer



Key Idea:

- Neural networks, like software modules, can be reused in appropriate scenarios.
- Early layers = general-purpose (edges, textures, embeddings)
- Head = task-specific (predicts original 1000 classes)

Example (ImageNet pre-trained CNN):

- Input → Conv layers → Pooling → FC layer
 - 1000 classes for ImageNet
 - Head: FC layer + softmax
- Example Task: Classify cats vs. dogs.
 - Use ImageNet
 - Steps:
 - Take pre-trained ResNet on ImageNet
 - Remove original head (1000 classes)
 - Add new head for 2 classes (cats, dogs)
 - Train only new head (or fine-tune some layers)

Example - Transfer Learning with ImageNet

```
# -----  
# 1. Load Pre-trained Base Model  
# -----  
# include_top=False removes the original head (ImageNet classifier)  
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224,224,3))  
  
# Freeze all base model layers initially  
for layer in base_model.layers:  
    layer.trainable = False  
  
# -----  
# 2. Attach New Head  
# -----  
x = base_model.output  
x = layers.GlobalAveragePooling2D()(x) # Flatten feature maps  
x = layers.Dense(128, activation='relu')(x) # Optional fully connected layer  
predictions = layers.Dense(2, activation='softmax')(x) # New head for 2 classes  
  
# Combine base and new head  
model = models.Model(inputs=base_model.input, outputs=predictions)  
  
# Compile model  
model.compile(optimizer=Adam(learning_rate=1e-3),  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])  
  
# -----  
# 3. Prepare New Dataset  
# -----  
# Assuming directory structure:  
# data/train/class1, data/train/class2  
train_datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)
```

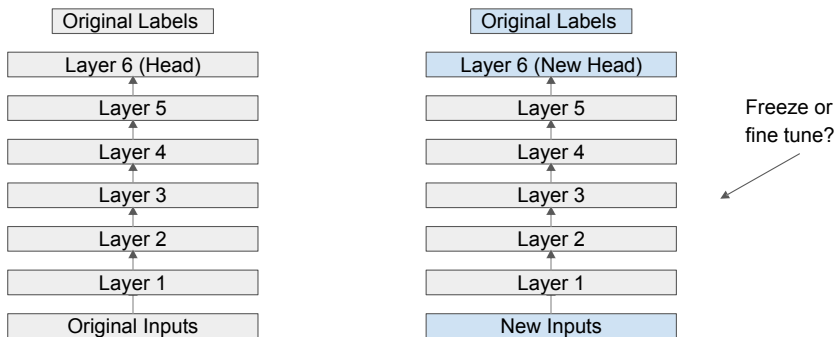
- ResNet50 is pre-trained on ImageNet.
 - 50-layer deep residual network
- base layers extract general visual features.
- head is replaced with a new task-specific classifier.

Step	What happens
1	Load ResNet50 pre-trained on ImageNet, remove original head
2	Add new fully connected layers + softmax for new classes
3	Prepare new dataset using <code>ImageDataGenerator</code>
4	Train only new head while base is frozen

```
train_generator = train_datagen.flow_from_directory(  
    'data/train',  
    target_size=(224,224),  
    batch_size=32,  
    class_mode='categorical',  
    subset='training'  
)  
  
val_generator = train_datagen.flow_from_directory(  
    'data/train',  
    target_size=(224,224),  
    batch_size=32,  
    class_mode='categorical',  
    subset='validation'  
)  
  
# -----  
# 4. Train the Model (Head Only)  
# -----  
model.fit(  
    train_generator,  
    epochs=5,  
    validation_data=val_generator  
)
```

Freeze vs Fine Tuning

When we import elements of the network, we can freeze their weights (not let them change) or allow them to be changed as part of the process of training on new data - **this is referred to as fine tuning.**



```
# -----  
# 5. Optional: Fine-Tune Last Layers of Base Model  
# -----  
# Unfreeze last 10 layers of the base for fine-tuning  
for layer in base_model.layers[-10:]:  
    layer.trainable = True  
  
# Recompile after changing trainable layers  
model.compile(optimizer=Adam(learning_rate=1e-4),  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])  
  
# Continue training  
model.fit(  
    train_generator,  
    epochs=5,  
    validation_data=val_generator  
)
```

Lab Work

- Check the notebook and go through it to see practical examples and review the theory as well.

APPENDIX